

Service now - ITOM

Discovery

What is ServiceNow Discovery?

ServiceNow Discovery is a tool that helps organizations automate the discovery and mapping of IT infrastructure and services across their network. It enables organizations to create a comprehensive, real-time map of their IT environment, including physical and virtual servers, databases, network devices, and applications.

ServiceNow Discovery uses a combination of active and passive techniques to gather data about devices and services on the network. Active discovery involves sending probes or requests to devices on the network to gather information about them, while passive discovery involves monitoring network traffic to identify devices and services.

The data collected by ServiceNow Discovery is used to populate the ServiceNow Configuration Management Database (CMDB), which serves as a single source of truth for IT infrastructure and services. This enables organizations to track changes and dependencies between IT components, identify potential issues, and improve their overall IT service management.

Overall, ServiceNow Discovery helps organizations to gain greater visibility into their IT infrastructure and services, reduce manual efforts, and improve the accuracy of their CMDB.

Discovery finds computers, servers, printers, a variety of IP-enabled devices, and the applications that run on them. It can then update the CIs in your CMDB with the data it collects.

Horizontal discovery and top-down discovery

There are two types of discovery:

Horizontal discovery

Horizontal discovery is a technique that Discovery uses to scan your network, find computers and devices, and then populate the CMDB with the CIs it finds. Horizontal discovery creates direct relationships between CIs, such as a runs on relationship between an application CI and the actual computer CI that it runs on. Horizontal discovery is not aware of business services and does not create relationships between CIs based on the business service they are in.

Top-down discovery

Top-down discovery is a technique that Service Mapping uses to find and maps CIs that are part of business services, such as an email service.

Example: Top-down discovery can map a website business service by showing the relationships between an Apache Tomcat web server service, a Windows server, and the MSSQL database that stores the data for the business service.

Typically, Service Mapping and Discovery work together to run horizontal discovery first to find CIs, and then top-down discovery to establish the relationships between business services that you need to know.

Probes, sensors, and patterns

Discovery uses these components to find CIs:

Probes and sensors

Probes and sensors are scripts that collect and process data on a host and then update the CMDB. More specifically, probes explore or investigate CIs on your network, and sensors parse the data returned from the probes. Several probes and sensors are provided by default, but you can customize them to find different information, or you can create ones. You can also configure several parameters to control the behaviour of a particular probe every time it is triggered.

Patterns

Patterns, like probes and sensors, are a series of operations that also collect data on a host, process it, and update the CMDB. Patterns differ from probes and sensors in that they are written in Neebula Discovery Language (NDL) rather than JavaScript, and they are called into action in the later stages of the horizontal discovery process. Default patterns are provided, but you can also customize or create patterns using the Pattern Designer.

Varshitha Education All Favorites History Workspaces Admin Servicenow Trainings- Probes ☆ Search

discovery

Range Sets
Behaviors
Functionality Definitions
Limited Resources
Simple Value
Based on Property
Scripted
MID Resources
Probes
Sensors
Port Probes
IP Services
Business Rules
Script Includes
Properties
CI Classification
All

≡ ▾ Probes Name Search

All > Used by Discovery = true

Name	ECC queue topic	ECC queue name	Description
AIX - Active Processes	SSHCommand	sh \${file:ps.sh}	Get (active) running processes
AIX - ADM	MultiProbe	AIX - Application Dependency Mapping	Queries for information about active (ru...
AIX - CPU	SSHCommand	sh \${file:cpus.sh}	Determine the OS processors
AIX - Hardware Model	SSHCommand	lsattr -El sys0 -a modelname -F value	Retrieve computer model
AIX - Identity	MultiProbe	AIX - Identity	Retrieve information for identifying it ...
AIX - Installed Software	SSHCommand	lspp -Lc	Retrieve AIX installed software
AIX - Memory	SSHCommand	sh \${file:memory.sh}	Retrieve memory information
AIX - Network	SSHCommand	/etc/ifconfig -au; netstat -in	Determine network interfaces, IPs, and MACs
AIX - OS Patches	SSHCommand	instfix -i grep AIX_ML	Retrieve OS patch info
AIX - OS Release	SSHCommand	oslevel	Retrieve OS version
AIX - OS Revision	SSHCommand	oslevel -r	Retrieve OS patch level
AIX - Serial Number	SSHCommand	echo `lsattr -El sys0 -a systemid cut ...	Retrieve serial number(s)
AIX - Active Connections	SSHCommand	sh \${file:aix_connection.sh}	Retrieve active connections information
Apache - Get Configuration	SSHCommand	sh \${file:get-apcfg.sh} -\${httpd_command...}	Retrieve Apache configuration information
Apache - Get JK Module	SSHCommand	sh \${file:get-jk-info.sh} \${cfg_location}	Retrieve load balancer information from ...

Varshitha Education All Favorites History Workspaces Admin Servicenow Trainings- Sensors ☆ Search

discovery

Range Sets
Behaviors
Functionality Definitions
Limited Resources
Simple Value
Based on Property
Scripted
MID Resources
Probes
Sensors
Port Probes
IP Services
Business Rules
Script Includes
Properties
CI Classification
All

≡ ▾ Sensors for text Search

All

Name	Sensor type	Reacts to probe	Description	Active
VMWare - Standalone ESXi Server	Javascript	VMWare - Standalone ESXi Server	Process ESXi Server. Extracts morids fo...	true
Linux - Find FQDN	Javascript	Linux - Find FQDN	Getting FQDN of a linux server	true
Windows - AWS Relationship	Javascript	Windows - AWS Relationship	Make relationship to cmdb_ci_ec2_instanc...	true
HTTP - Classify	Javascript	HTTP - Classify	HTTP - Classify sensor	true
VMWare - vCenter ESX Hosts Storage	Javascript	VMWare - vCenter ESX Hosts Storage	Process ESX host storage data.	true
Windows - WebSphere - Web Services	Javascript	Windows - WebSphere - Web Services	Map contents of server.xml into WebSpher...	true
Solaris - Installed Software	Javascript	Solaris - Installed Software	Process Installed Software	true
Windows - Azure	Javascript	Windows - Azure	Determines whether this Windows machine...	true
Shazzam	Javascript	Shazzam	Processes the results of a Shazzam probe.	true
UNIX - Classify	Javascript	UNIX - Classify	Process information to classify the type...	true
SNMP - DRAC	Javascript	SNMP - DRAC	SNMP - DRAC Sensor	true

Horizontal discovery phases

The phases of horizontal discovery are:

Discovery follows these phases:

Scanning

Discovery sends a probe called *Shazzam* to the network to see if commonly used ports are open and if these ports can respond to queries. For example, if Shazzam finds a device that responds on port 135, Discovery knows that it is a Windows server.

Classification

If Discovery finds devices or computers, it sends additional probes to find the type of device or the operating system on the device. For example, Discovery sends the WMI probe to a Windows machine to detect the Windows 2012 operating system. Then Discovery uses records called *classifiers*, which specify the trigger probe or probes that run during the next two phases. If you are using patterns, the classifier specifies a trigger probe that in turn launches a pattern.

Identification

Discovery tries to gather more information about the device and then tries to determine if a CI for the device exists in the CMDB. Discovery then uses additional probes, sensors, and identifiers to update existing CIs in the CMDB or create new ones. *Identifiers*, also known as identification rules, specify the attributes that the probes look at when reconciling data with the CIs in the CMDB. If you are using patterns, Discovery uses the appropriate identification rule for the CI type specified in the pattern.

Exploration

The identifier launches additional probes configured in the classifier. These probes are especially designed as exploration probes to gather additional information about the device, like the applications running it, and additional attributes, such as memory, network cards, and drivers. Discovery then creates relationships between applications and devices and between applications. If you are using patterns, the operations in the pattern perform the exploration of the CI.